

Product Requirements Document (PRD)

ListaPro

Parte 3 — Arquitetura Técnica

14. Arquitetura Geral

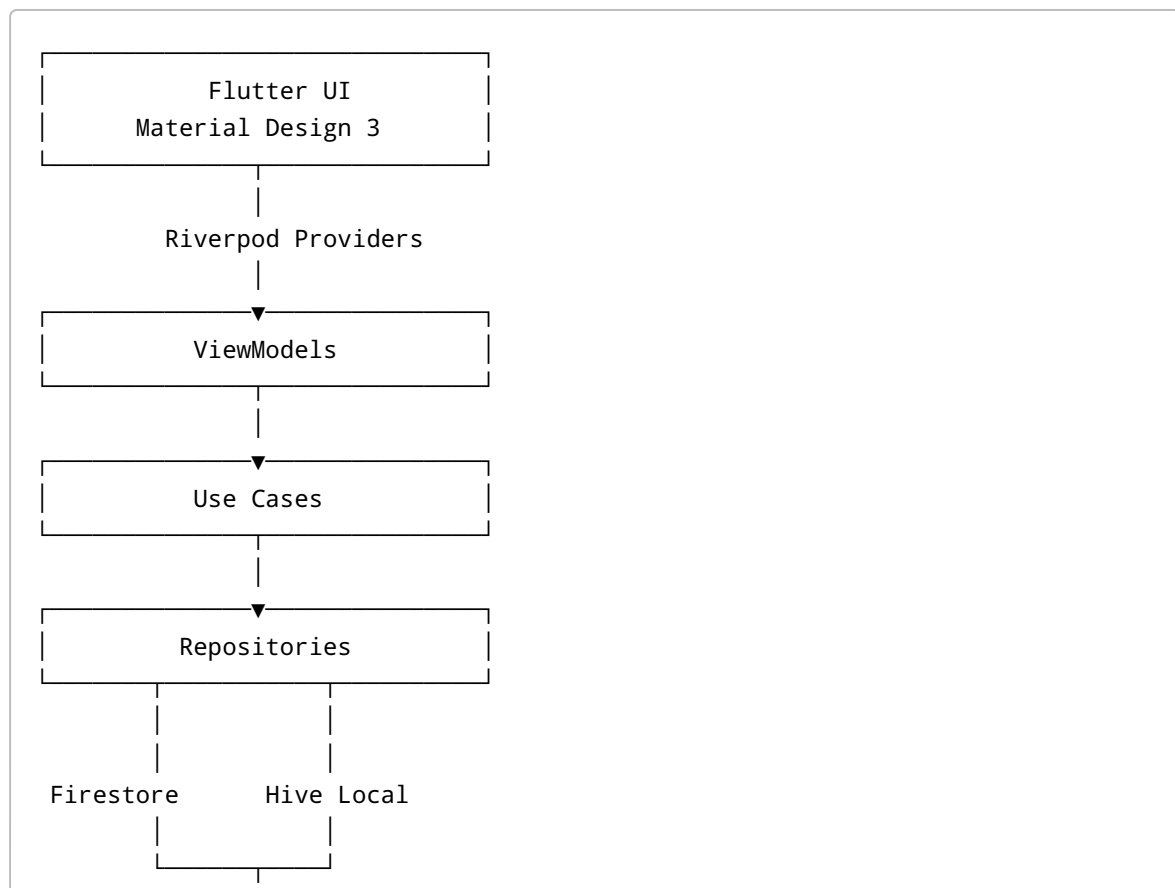
Visão Geral

O aplicativo será desenvolvido utilizando uma arquitetura moderna, escalável e desacoplada.

A arquitetura deverá seguir os princípios da Clean Architecture, separando completamente regras de negócio, interface e acesso aos dados.

A estrutura permitirá futuras expansões sem necessidade de reescrever componentes principais.

Arquitetura



15. Stack Tecnológica

Front-end

Flutter

Linguagem

Dart

Arquitetura

Clean Architecture

Design Pattern

MVVM

Estado

Riverpod

Navegação

GoRouter

Banco Online

Cloud Firestore

Banco Offline

Hive

Autenticação

Firebase Authentication

Push

Firebase Cloud Messaging

Analytics

Firebase Analytics

Logs

Firebase Crashlytics

Versionamento

Git

GitHub

GitHub Actions

16. Estrutura do Projeto

```
lib/  
  
core/  
  
config/  
  
constants/  
  
errors/  
  
extensions/  
  
services/  
  
theme/
```

```
utils/  
features/  
authentication/  
shopping_lists/  
dashboard/  
history/  
profile/  
notifications/  
shared/  
widgets/  
main.dart
```

Estrutura Interna de uma Feature

Cada módulo deverá seguir exatamente esta organização.

```
shopping_lists/  
data/  
datasources/  
models/  
repositories/  
domain/  
entities/  
repositories/  
usecases/  
presentation/
```

```
pages/  
widgets/  
viewmodels/  
providers/
```

17. Modelagem do Banco

Firestore

Coleções principais

```
users  
  
shopping_lists  
  
shopping_items  
  
notifications  
  
audit_logs
```

Coleção Users

Campos

```
{  
  "id": "uuid",  
  "name": "Carlos",  
  "username": "carlos",  
  "role": "ADMIN",  
  "active": true,  
  "createdAt": "...",  
  "updatedAt": "..."  
}
```

Role

Valores possíveis

ADMIN

EMPLOYEE

Coleção Shopping Lists

```
{
  "id": "uuid",
  "title": "Compras Mercado",
  "description": "...",
  "priority": "HIGH",
  "status": "PENDING",
  "createdBy": "userId",
  "assignedTo": "adminId",
  "createdAt": "...",
  "updatedAt": "...",
  "finishedAt": null,
  "version": 1,
  "offlineChanges": false
}
```

Status

Valores

DRAFT

PENDING

IN_PROGRESS

FINISHED

CANCELLED

Prioridade

LOW

MEDIUM

HIGH

Shopping Items

```
{
  "id": "uuid",
  "listId": "uuid",
  "name": "Arroz",
  "quantity": 2,
  "unit": "KG",
  "brand": "Tio João",
  "category": "Mercearia",
  "expectedPrice": 25.90,
  "notes": "",
  "checked": false,
  "checkedBy": null,
  "checkedAt": null,
  "position": 1,
  "version": 1
}
```

Notifications

```
{
  "id": "uuid",
  "userId": "uuid",
  "title": "Nova Lista",
  "message": "Você recebeu uma nova lista.",
  "read": false,
  "createdAt": "..."
}
```

Audit Logs

Toda ação relevante deverá gerar um log.

```
{
  "id": "uuid",
  "userId": "uuid",
  "action": "CHECK_ITEM",
  "entity": "SHOPPING_ITEM",
}
```

```
"entityId": "uuid",  
"device": "Android",  
"timestamp": "..."  
}
```

18. Relacionamentos

```
User  
|  
├─ cria  
|  
ShoppingList  
|  
├─ possui  
|  
ShoppingItem  
|  
├─ gera  
AuditLog
```

19. Sincronização

O sistema deverá utilizar sincronização em tempo real através do Firestore.

Sempre que houver alteração:

↓

Firestore atualiza documento

↓

Snapshot Listener dispara

↓

Repository recebe alteração

↓

ViewModel atualiza estado

↓

Interface atualiza automaticamente

Fluxo Offline

Sem internet

↓

Salvar alteração no Hive

↓

Adicionar alteração na fila

↓

Internet retorna

↓

Enviar fila

↓

Firestore confirma

↓

Remover fila

↓

Atualizar interface

Controle de Versão

Toda alteração incrementará

version +1

Exemplo

Lista

Versão 1

↓

Adicionar Item

Versão 2

↓

Editar Item

Versão 3

↓

Finalizar

Versão 4

UUID

Todos os objetos deverão possuir UUID v4.

Nunca utilizar IDs sequenciais.

Timestamp

Todos os registros possuirão

createdAt

updatedAt

Opcionalmente

deletedAt

finishedAt

checkedAt

20. Segurança

Toda comunicação deverá utilizar HTTPS.

Nenhuma senha será armazenada localmente.

Sessões deverão utilizar Token Firebase.

As permissões serão controladas por Role.

Funcionário:

Não poderá visualizar listas de outros funcionários.

Administrador:

Visualiza todas.

21. Regras do Firestore

Shopping Lists

Funcionário

Pode:

Criar

Editar (antes do envio)

Consultar próprias listas

Administrador

Pode:

Consultar todas

Atualizar status

Finalizar

Shopping Items

Funcionário

Pode criar

Pode editar

Pode excluir

Antes do envio.

Administrador

Pode marcar

Pode desmarcar

Após envio.

Users

Somente administrador do sistema.

Audit Logs

Somente escrita automática.

Nunca editáveis.

22. Performance

Objetivos

Login

< 2 segundos

Abrir Home

< 2 segundos

Abrir Lista

< 1 segundo

Atualizar Item

< 300 ms

Sincronização

Instantânea quando online

23. Escalabilidade

A arquitetura deverá permitir futuramente:

Múltiplos compradores

Scanner

Fotos

Código de barras

Controle financeiro

Estoque

Relatórios

Dashboard

IA

Integração com ERP

Sem necessidade de alterar a estrutura principal do banco de dados.

24. Dependências Flutter

Principais pacotes previstos

```
flutter_riverpod  
go_router  
firebase_core  
firebase_auth  
cloud_firestore  
firebase_messaging  
firebase_crashlytics  
firebase_analytics  
hive  
hive_flutter  
connectivity_plus  
flutter_secure_storage  
uuid  
intl  
freezed  
json_serializable  
equatable  
dio  
logger
```

Considerações Técnicas

Todo o projeto deverá seguir princípios SOLID.

Nenhuma regra de negócio poderá ficar na camada de interface.

Toda comunicação com Firebase deverá ocorrer exclusivamente através dos Repositories.

Todas as telas deverão consumir apenas ViewModels.

Não será permitido acesso direto ao Firestore dentro das telas.

Todas as entidades deverão ser imutáveis.

Todo código deverá ser documentado e seguir boas práticas de legibilidade e organização.

Fim da Parte 3